

Teman Tanyamu (dash_ai_me)

Lightweight self-hosted AI chat web app. An Open WebUI alternative with **under 100 MB RAM**. No Docker. No build step. One Node.js process + one SQLite file.

Stack	Node.js + Express 5 + better-sqlite3 + vanilla JS
AI Backend	9Router (<code>http://localhost:20128/v1</code> , OpenAI-compatible)
Providers	AG, Groq, BAI, OpenRouter, Nvidia, Cline — 497+ models
Frontend	<code>index.html</code> + <code>style.css</code> + <code>app.js</code> , zero framework, dark theme
Language	English (default) / Indonesian — runtime toggle
Version	v1.0-beta
License	Private / personal use

Table of Contents

- 1. [Features](#)
- 2. [System Architecture](#)
- 3. [Request Flow Diagrams](#)
- 4. [Requirements](#)
- 5. [Installation](#)
- 6. [Management Script](#)
- 7. [Configuration](#)
- 8. [API Reference](#)
- 9. [Database Schema](#)
- 10. [Access from Other Devices](#)
- 11. [Performance](#)
- 12. [Security Notes](#)
- 13. [Troubleshooting](#)
- 14. [Changelog](#)

Features

Chat

Feature	Description
Real-time streaming	SSE relay, word-by-word from 9Router
Multi-session history	Per-user isolation in SQLite

Feature	Description
Chat memory	Token-budget context window (default 1600 tokens ≈ 15–30 msgs, oldest auto-compressed) + rolling background summary of older turns
Cross-chat memory	Auto-extracted personal facts per user (max 20), injected into every chat; editable in Settings → Memory
Token saver	<code>max_tokens</code> reply cap (default 1024), history budget, per-chat Lean ⚡ toggle (answers without history)
Per-reply usage badge	prompt / completion / total tokens shown under each AI reply
Rename / Delete chats	Via sidebar hover buttons; rename uses custom in-app dialog (bilingual, no native prompt)
Pin / Archive / Tag / Fork	Chat "...": pin to top, archive to a collapsible "Archived (n)" section, inline #tag, fork = branch copy of the conversation ((fork) suffix)
Chat search	Instant client-side filter by title (sidebar)
Markdown + code blocks	With streaming auto-close fence fix
Copy & Retry	Per-message actions
Smart chat title	After the first reply a short 3–6 word title is generated in the background (never overwrites a manual rename)
Edit & regenerate	Pencil on any user message → inline edit → save truncates everything after it; ↺ button regenerates from the last user message
Code highlight + copy	fenced blocks get language colouring (bundled hljs, no CDN) + per-block copy button
Stop generation	Abort mid-stream
Provider timeout	Admin-configurable 30–600 s (default 120 s), applies to all upstream calls
Typing indicator	Animated dots while waiting
Temporary Chat	Zero history saved (endpoint: <code>/api/temp-chat</code>); honours Lean mode (skip history + memory = zero context)
Realtime quota UI	Sidebar quota + admin dashboard refresh automatically after every reply (dashboard auto-polls every 5 s while open)

Users & Access

Feature	Description
Login	script per-user salt (<code>script1:salt:hash</code>), timing-safe compare; legacy static-salt hashes auto-upgrade on login
Session purge	Expired sessions deleted on every login
Roles	<code>admin</code> (full control) and <code>user</code> (chat only)
Daily quota	Per-user limit (default 50/day), admin unlimited; quota bar shown in sidebar with model chip
User management	Admin: add / delete / reset password / activate / deactivate (self-deletion & self-deactivation blocked server-side)
Instant session revoke	Deactivate or password reset kills sessions immediately

Feature	Description
Quota edit & reset	Admin edits any user's daily quota (1–999999) and resets one or all users' usage

Guest Mode (no login)

Feature	Description
Try as guest	Button under Login on the login screen (only shown when guest mode is enabled + gateway configured); clicks straight into the chat dashboard
Configurable limits	Admin sets max messages per guest (1–200, default 10) and session time limit (1–60 min, default 5)
Live guest chip	Sidebar shows <code>Guest — n/max messages · mm:ss left</code> (turns red under 60 s)
Login popup on exhaustion	When chats or time run out, an in-app modal asks to sign in; "Continue as guest" dismisses it, "Sign in now" goes to login
Server-enforced	Limits checked before the AI call; rejected sends roll back their bubbles (nothing stored); expired sessions can't post to any account endpoint (<code>ERR_GUEST_NOPE</code>)
Session persistence	Cookie <code>gtoken</code> (HttpOnly, 24 h); chat history kept during the session, survives reload; re-clicking guest mints a fresh session
Clean stats	Guest messages never touch the <code>usage</code> table — admin statistics stay per-real-user
Auto-delete (nightly purge)	Scheduler wipes ALL guest data — sessions, chats, messages — once per day at an admin-configurable local hour (default 00:00, <code>guest_purge_hour</code>); plus a manual "Purge guest chats now" button in Admin → Usage Statistics

Chat Experience

Feature	Description
Smart title	After the first reply, a short 3–6-word title is generated in the background (never overwrites manual renames; guests excluded)
Edit message	Pencil on user bubbles → inline textarea → save = rewrite + truncate everything after, resend automatically
Regenerate	↻ on AI bubbles truncates from the last user message and resends
Code highlighting	Fenced blocks coloured via bundled highlight.js (v11.9, served from <code>/vendor/</code> , no CDN) + per-block copy button; theme-aware (github-dark/github swap with the UI theme)
Pin / Tag / Fork / Archive	Chat "..." menu — pin keeps chats at top, <code>#tag</code> shown as chip, fork copies the whole conversation as <code>(fork)</code> , archive hides chats into a collapsible "Archived (n)" section

Usage Statistics (Admin)

Last-30/60/90-days dashboard: card row (requests, total/prompt/completion tokens, active users, guest sessions), a canvas bar chart of daily tokens (no chart library, re-themes with light/dark), and top-10 bars by model and by user. Endpoints: `GET /api/admin/stats?days=30` , `POST /api/admin/guest-purge-now` .

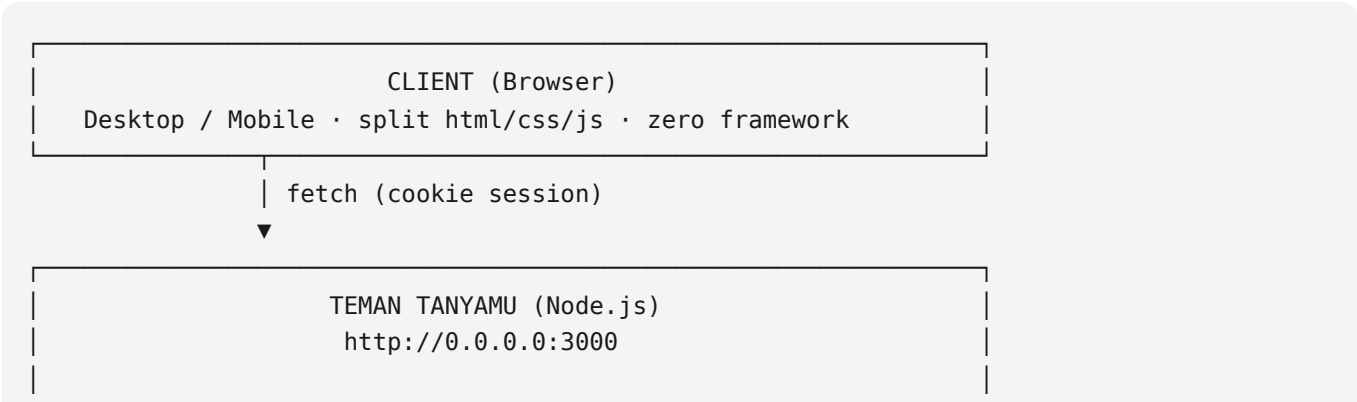
Models

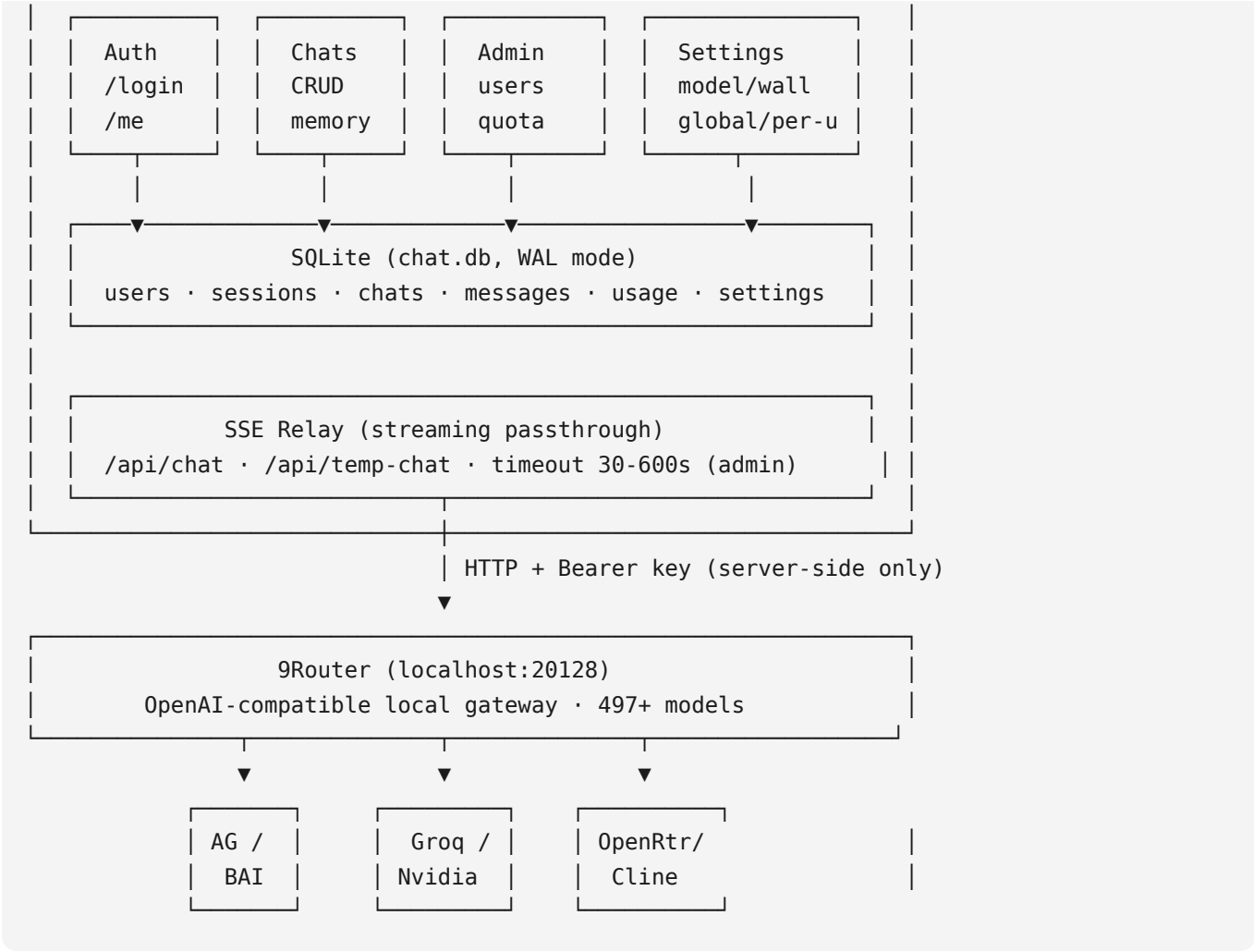
Feature	Description
Model Connection (gateway)	Base URL + API key set in Admin Dashboard (Open WebUI style): masked key display, show/hide eye, Test button (live latency + model count), instant hot-reload, no restart
Auto-fetch	Model list pulled live from 9Router (or any OpenAI-compatible endpoint)
Searchable picker	Custom combobox with instant filter + keyboard nav (↑/↓/Enter/Esc), works for admin & locked users
Global default	Admin setting, applies to all users
Per-user override	Admin sets specific model per user
Per-chat model	Each chat remembers its model
Locked for users	Regular users cannot change model (server-enforced); UI shows a red "Terkunci / Locked" chip

Appearance

Feature	Description
Light / dark theme	Toggle in the sidebar (sun/moon); dark = Open WebUI palette, light derived from it. Persisted per browser; highlight.js CSS theme swaps with it
Chat wallpaper	6 presets + custom image upload (per-user, optional global); dark overlay only for image wallpapers, gradient presets render as-is
Custom avatar	Auto-downscaled upload, strict base64 validation; rendered in chat bubbles, sidebar and admin list
Global assistant avatar	Admin uploads one AI avatar for everyone (<code>/api/admin/assistant-avatar</code>), or resets to default mask icon
Admin avatar popover	Click any avatar row in the admin user table → upload / reset / cancel (no native prompts anywhere)
Sidebar minimize	Desktop: icon rail (user-row collapses to avatar + stacked actions), state persisted · Mobile: closes the drawer
Bilingual UI	English / Indonesian, all strings translated — dialogs, toasts, admin panel, error codes
Custom dialogs	All confirm/rename/quota dialogs are in-app modals (i18n + consistent style), never native <code>alert/confirm/prompt</code>
Responsive	Mobile drawer sidebar, 44px+ tap targets, admin tables become card stacks, verified 1920→320 px with no horizontal overflow

System Architecture



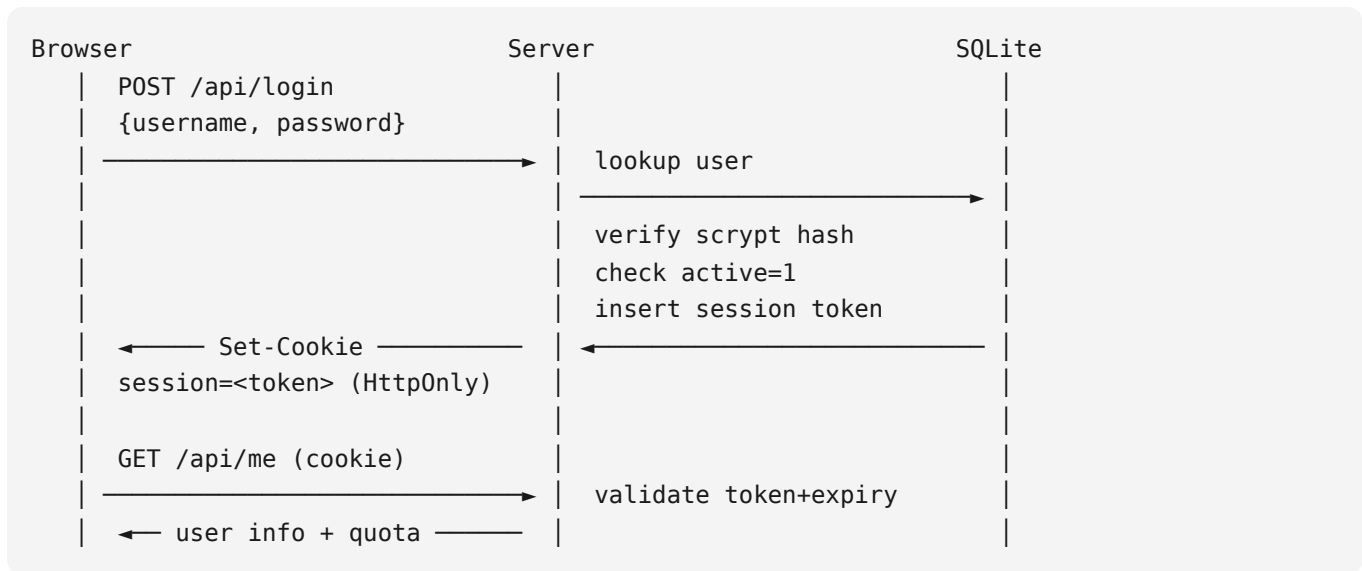


Key Design Decisions

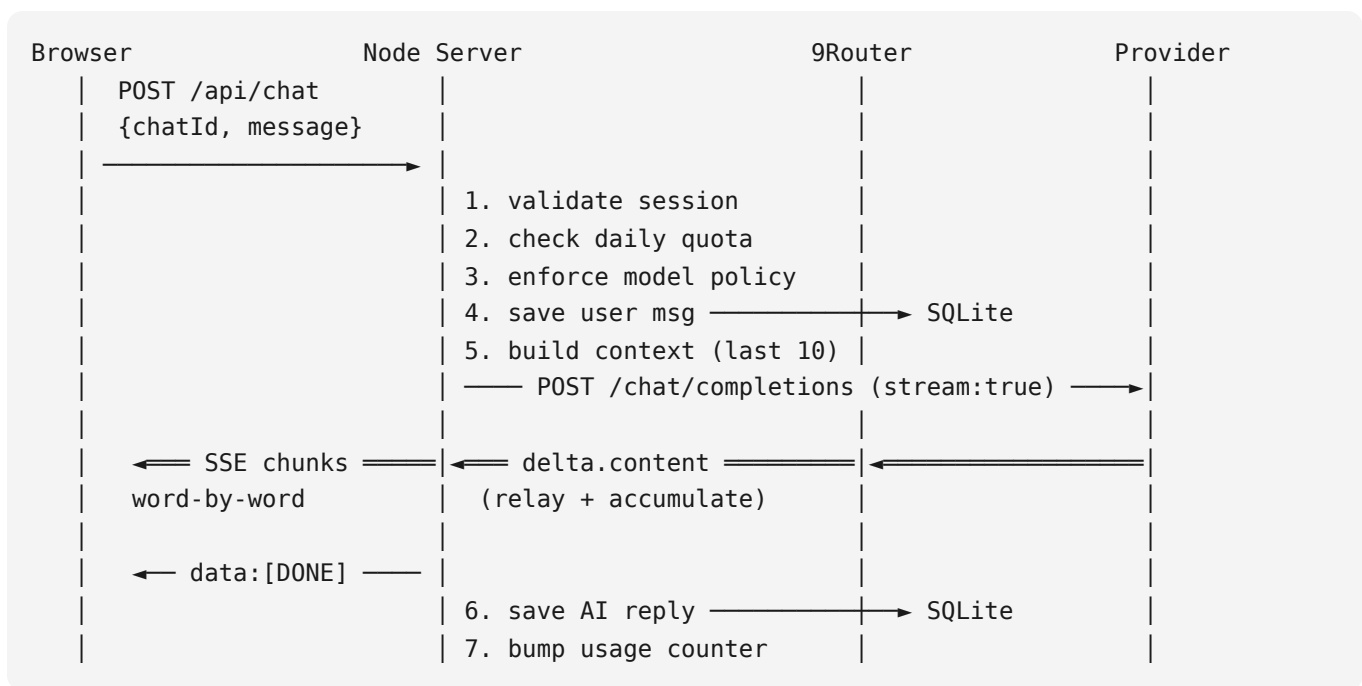
Decision	Reason
Single file backend (<code>server.js</code>)	Easy audit, zero complexity
SQLite over Postgres	Zero-config, WAL mode, single file backup
Server-side API key proxy	Key never reaches the browser
SSE relay (not direct)	Streaming works cross-device, CORS-free
Error codes (<code>ERR_*</code>)	Translated client-side per language
no-cache delivery + versioned assets	HTML: no-store · CSS/JS: no-cache + <code>?v=</code> query (updates always fresh, stale caches bust on release)
Split frontend (<code>style.css</code> + <code>app.js</code>)	Cacheable assets, lintable JS, CSP-ready

Request Flow Diagrams

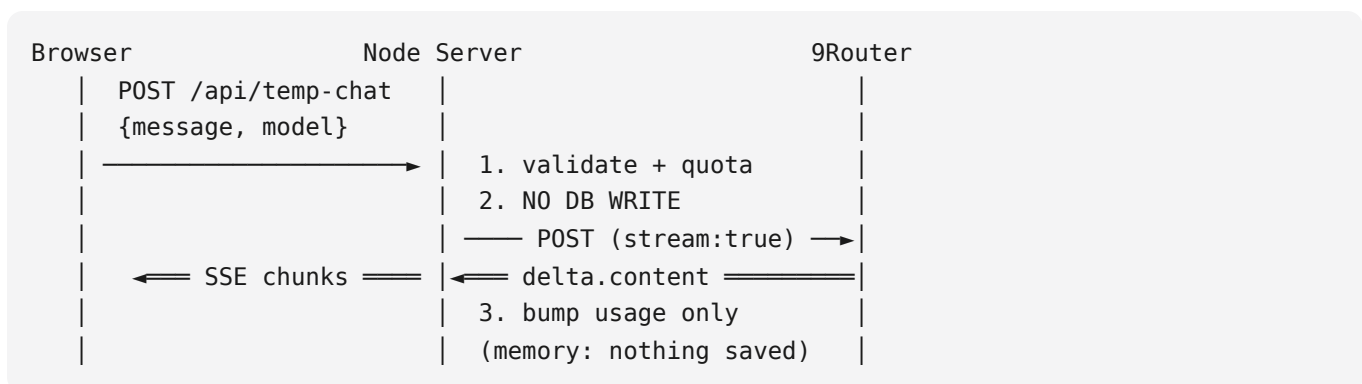
1. Login Flow



2. Streaming Chat Flow



3. Temporary Chat Flow



4. Admin Model Policy

User (non-admin) changes model?

- UI: selector disabled
- Server (double check):
 - PUT /api/me/model → 403 ERR_ADMIN_ONLY
 - PATCH /api/chats model → 403 ERR_ADMIN_ONLY
 - POST /api/chats {model} → forced to resolveUserModel()
 - POST /api/chat → chat.model auto-corrected

Admin sets global default → settings.default_model

Admin sets user override → users.model_override

Resolution order: user override > global default

Requirements

Component	Minimum
OS	Linux (tested: Ubuntu 24.04)
Node.js	>= 20 (tested: v24.21)
9Router	Running, OpenAI-compatible, default port 20128
Disk	~50 MB (incl. node_modules)

Installation

cd dash_ai_me
npm install # express + better-sqlite3
node server.js # http://localhost:3000

First run creates the admin account automatically:

username: admin
password: admin123 # change immediately via UI

Management Script (`manage.sh`)

All-in-one service manager (systemd user service):

```
./manage.sh start          # Start service
./manage.sh stop           # Stop service
./manage.sh restart        # Restart service
./manage.sh status         # Status, port, RAM
./manage.sh port <1-65535> # Change port (persistent, auto-restart)
./manage.sh clear-cache    # Clean SQLite WAL + npm cache + old journal
./manage.sh enable-boot    # Auto-start at boot (enables linger)
./manage.sh disable-boot   # Disable boot autostart
./manage.sh logs [n]       # Last n log lines (default 50)
./manage.sh password       # Reset admin password to admin123
./manage.sh help           # Show help
```

`enable-boot` runs `loginctl enable-linger` so the service runs without an active desktop session.

Configuration

Environment Variables

Variable	Default	Description
<code>PORT</code>	<code>3000</code>	Web server port
<code>ROUTER_BASE</code>	–	Boot fallback base URL (used only until set in Admin Dashboard)
<code>ROUTER_KEY</code>	–	Boot fallback API key (same; DB settings take precedence)
<code>ADMIN_PASSWORD</code>	<code>admin123</code>	Initial admin password (first run only)

Example:

```
PORT=8080 ROUTER_BASE=http://192.168.1.10:20128/v1 ROUTER_KEY=*** node server.js
```

Once set via **Admin Dashboard** → **Model Connection**, values in the `settings` table win over env vars — you can change gateway/key anytime from the browser without restart.

File Locations

Path	Description
<code>chat.db</code>	SQLite database (WAL mode)
<code>public/index.html</code>	Struktur halaman SPA (login + app shell)
<code>public/style.css</code>	Seluruh styling frontend
<code>public/app.js</code>	Seluruh logika frontend
<code>server.js</code>	Backend (single file)
<code>.port</code>	Persisted port number (created by <code>manage.sh port</code>)

Path	Description
<code>~/.config/systemd/user/dash-ai-me.service</code>	systemd unit

API Reference

All endpoints require the session cookie except `/api/login` and `/api/health`. Errors return `{"error": "ERR_*"}` codes, translated client-side.

Auth

Method	Endpoint	Body	Description
POST	<code>/api/login</code>	<code>{username, password}</code>	Sign in
POST	<code>/api/logout</code>	-	Destroy session (user + guest cookies)
GET	<code>/api/me</code>	-	User info + quota + effective model + global assistant avatar (guest: <code>guest</code> , <code>quota_used/max</code> , <code>seconds_left</code> , <code>expired</code>)
POST	<code>/api/guest</code>	-	Start/reuse guest session (cookie <code>gtoken</code> ; 403 when guest mode off)
GET	<code>/api/guest-config</code>	-	<code>{enabled}</code> — login screen show/hide for the guest button (no auth)
GET	<code>/api/health</code>	-	Liveness + DB check (no auth, for uptime monitors)

Account

Method	Endpoint	Body	Description
PUT	<code>/api/me/password</code>	<code>{current, next}</code>	Change own password
PUT	<code>/api/me/avatar</code>	<code>{avatar: dataURL null}</code>	Set/remove avatar
PUT	<code>/api/me/model</code>	<code>{model}</code>	Own model override (admin)
DELETE	<code>/api/me/model</code>	-	Clear override (admin)

Chats

Method	Endpoint	Body	Description
GET	<code>/api/chats</code>	-	List own chats
POST	<code>/api/chats</code>	<code>{title?, model?}</code>	Create chat
GET	<code>/api/chats/:id</code>	-	Chat + full messages
PATCH	<code>/api/chats/:id</code>	<code>{title?, model?, pinned?, archived?, tag?}</code>	Rename / lock model (admin) / pin / archive / tag

Method	Endpoint	Body	Description
POST	/api/chats/:id/fork	-	Copy chat + messages into a new (fork) chat
PUT	/api/chats/:id/messages/:msgId	{content}	Edit message content (user messages only)
DELETE	/api/chats/:id/messages/:msgId	-	Truncate chat: delete that message + everything after it
PATCH	/api/chats/:id/lean	{lean: bool}	Toggle lean (no-history) token saver per chat
DELETE	/api/chats/:id	-	Delete chat + messages

Memory (cross-chat, per user)

Method	Endpoint	Body	Description
GET	/api/memory	-	List own memory facts + enabled flag
PUT	/api/memory	{facts: string[]}	Replace full fact list (max 20, 240 chars each)
DELETE	/api/memory/:factId	-	Delete one fact
POST	/api/memory/clear	-	Wipe own memory

Chat & Preferences

Method	Endpoint	Body	Description
GET	/api/models	-	Model list from 9Router
POST	/api/chat	{chatId, message}	Streamed chat (SSE); token-budget context + system memory injected; final usage event with prompt/completion split
POST	/api/temp-chat	{message, model?, lean?}	Streamed temp chat (SSE, not saved); lean:true = zero context (no system prompt/memory)
GET	/api/wallpaper	-	Effective wallpaper
PUT	/api/wallpaper	{wallpaper null}	Own wallpaper

Admin

Method	Endpoint	Body	Description
GET	/api/admin/users	-	Users + today's usage + effective model + me id
POST	/api/admin/users	{username, password, role, daily_quota}	Create user
DELETE	/api/admin/users/:id	-	Delete user (self-delete blocked)
PUT	/api/admin/users/:id/password	{password}	Reset password (revokes sessions)
PUT	/api/admin/users/:id/active	{active: bool}	Activate / deactivate (self-deactivate blocked)

Method	Endpoint	Body	Description
PUT	/api/admin/users/:id/quota	{daily_quota}	Edit daily quota (1–999999)
PUT	/api/admin/users/:id/reset-quota	-	Reset one user's daily usage
PUT	/api/admin/reset-usage	-	Reset all users' usage (DELETE FROM usage)
PUT	/api/admin/users/:id/model	{model null}	Per-user model override
POST	/api/admin/avatar	{userId, avatar null}	Upload / reset any user's avatar (self-sync immediate)
POST	/api/admin/assistant-avatar	{avatar null}	Global assistant (AI) avatar for all users
GET	/api/admin/stats	?days=7 30 90	Usage statistics: totals, per-day series, top models/users, guest sessions
POST	/api/admin/guest-purge-now	-	Purge all guest data immediately
GET	/api/admin/settings	-	Global settings + today's token totals (prompt/completion)
PUT	/api/admin/settings	{default_model?, history_token_budget?, max_reply_tokens?, memory_enabled?, timeout_ms?, guest_enabled?, guest_max_chats?, guest_max_minutes?, guest_purge_hour?}	Update settings (budget 200–32000, reply cap 64–8192, timeout 30–600 s, guest chats 1–200, guest minutes 1–60, purge hour 0–23)
GET	/api/admin/router-config	-	Model gateway base URL + masked key + source (database / env)
PUT	/api/admin/router-config	{base_url?, api_key?}	Set gateway base URL & API key (hot-reload, no restart; empty key = keep)
POST	/api/admin/router-test	{base_url?, api_key?}	Probe GET {base}/models with saved or posted (pre-save) creds → latency + model count
PUT	/api/admin/wallpaper	{wallpaper null}	Set global wallpaper

Error Codes

Code	Meaning
ERR_UNAUTH	Not signed in

Code	Meaning
ERR_ADMIN_ONLY	Admin-only action
ERR_FIELDS_REQUIRED	Missing fields
ERR_LOGIN	Wrong credentials
ERR_INACTIVE	Account deactivated
ERR_PW_SHORT	Password under 4 chars
ERR_OLD_PW	Old password wrong
ERR_QUOTA	Daily quota exceeded
ERR_RATE_LIMITED	Too many failed logins: 5 attempts, then 2-minute IP block
ERR_NOT_FOUND	Resource not found
ERR_USERNAME_TAKEN	Username exists
ERR_TOO_BIG	Image too large
ERR_AVATAR_FORMAT	Avatar not an image
ERR_WALLPAPER_FORMAT	Wallpaper format invalid
ERR_SELF_DELETE	Cannot delete own account
ERR_SELF_DEACTIVATE	Cannot deactivate own account
ERR_BAD_BODY	Malformed JSON body
ERR_BAD_BUDGET	History budget out of range (200–32000)
ERR_BAD_MAXTOK	Reply token cap out of range (64–8192)
ERR_BAD_TIMEOUT	Upstream timeout out of range (30–600 s)
ERR_ROUTER_DOWN	9Router unreachable
ERR_ROUTER_NOT_CONFIGURED	Base URL / API key not set yet (Admin Dashboard → Model Connection)
ERR_BASE_REQUIRED	Base URL field empty
ERR_BASE_INVALID	Base URL not a valid http(s) URL
ERR_BASE_BLOCKED_HOST	Base URL host blocked (link-local / metadata addresses)
ERR_TIMEOUT	Upstream model call timed out
ERR_GUEST_DISABLED	Guest mode is turned off
ERR_GUEST_CHATS	Guest message limit reached
ERR_GUEST_TIME	Guest session time limit reached
ERR_GUEST_NOPE	Account feature unavailable in guest mode

Database Schema

```
users      (id, username, password_hash, role, daily_quota,
           model_override, avatar, active, created_at)
```

```

-- password_hash: "script1:<salt>:<hash>" (per-user salt);
-- legacy static-salt hex auto-upgrades on next login
sessions (token, user_id, expires_at) -- expired rows purged on login
chats (id, user_id, title, model, lean, pinned, archived, tag,
      summary, created_at, updated_at) -- lean: 1 = no-history token saver
-- pinned/archived: 0/1; tag: free text (<= 24 chars)
messages (id, chat_id, role, content, tokens, created_at)
usage (user_id, date, request_count, tokens_used) -- PK (user_id, date)
guest_sessions (token, guest_id, msgs_used, started_at, expires_at)
-- guest chats stored under user_id = -guest_id; wiped on logout / purge
memories (id, user_id, content, source, updated_at) -- cross-chat facts (max 20/user)
settings (key, value)
-- keys: default_model, wallpaper, assistant_avatar, router_base, router_key,
--       user:<id>:wallpaper, history_token_budget, max_reply_tokens,
--       memory_enabled, timeout_ms, memory_sync_chat, memory:last:<id>,
--       guest_enabled, guest_max_chats, guest_max_minutes, guest_purge_hour

```

Access from Other Devices

Server binds to all interfaces. From phones / LAN devices:

```
http://<laptop-ip>:3000          e.g. http://20.20.100.6:3000
```

For access over the internet, a Cloudflare Tunnel works out of the box: `trust proxy` is enabled and `cf-connecting-ip` is honored for login rate limiting; the session cookie gets the `Secure` flag automatically when the request arrives via HTTPS.

Performance

Metric	Measured
RAM idle	20-30 MB
RAM peak (streaming)	50-65 MB
CPU idle	~0%
Boot time	< 3 s
Streaming first token	~0.5-2 s (provider dependent)

Security Notes

- Passwords: script with per-user random salt, stored as `script1:<salt>:<hash>`, verified with `crypto.timingSafeEqual`; legacy static-salt hashes upgrade automatically on the next successful login

- Sessions: HttpOnly cookie, SameSite=Lax, 7-day expiry; expired rows purged on login
- **Secure cookie flag**: auto-enabled when behind HTTPS (Cloudflare Tunnel sets `X-Forwarded-Proto`)
- **Login rate limiting**: 5 failed attempts per IP, then a 2-minute block (`ERR_RATE_LIMITED`)
- `trust proxy` enabled: client IP correctly detected behind Cloudflare (`CF-Connecting-IP`)
- **Security headers** on every response: `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy: no-referrer`
- **XSS hardening**: avatar & wallpaper validated server-side against strict formats (base64 image types / `linear-gradient(...)` / `#hex` only — no `svg+xml` , no attribute breakout); all user-supplied values escaped at render
- 9Router API key never sent to browser (server-side proxy only)
- Per-user chat isolation enforced in every SQL query
- Model policy enforced server-side (not just hidden in UI)
- Guest mode: chat/time limits enforced server-side **before** the AI call; guests blocked from all account endpoints (`ERR_GUEST_NOPE`); guest chats isolated under negative user ids and excluded from the `usage` stats table
- Quota checked before every AI call; failed/aborted streams do not consume quota
- Deactivate / password reset revokes all sessions instantly
- Caching: HTML `no-store` , CSS/JS `no-cache` + `?v=` asset version in `index.html` (bump on each release so old client caches invalidate automatically)

Troubleshooting

Problem	Fix
"9Router offline"	Verify gateway: <code>curl http://localhost:20128/v1/models -H "Authorization: Bearer ***"</code> — or click Test in Admin Dashboard → Model Connection
Moved gateway / new key	Admin Dashboard → Model Connection: set Base URL + API key, Test, Save. Applies instantly, no restart
Forgot admin password	<code>./manage.sh password</code>
Port already in use	<code>./manage.sh port 3001</code>
Not auto-starting at boot	<code>./manage.sh enable-boot</code> , verify <code>loginsctl show-user \$USER grep Linger</code>
Stale UI after update	Hard refresh: <code>Ctrl+Shift+R</code> (HTML/CSS/JS are all no-cache; rarely needed)
Full reset (nuclear)	<code>./manage.sh stop && rm -f chat.db* && ./manage.sh start</code> — all data gone

Changelog

v1.0-beta (current)

- **Chat Experience** — smart auto-title after first reply, inline edit & regenerate (truncate + resend), highlight.js code blocks with copy button, pin / tag / fork / archive chat menu
- **Light / dark theme** — sidebar toggle, persisted per browser, hljs + stats chart re-theme live
- **Usage Statistics (admin)** — 7/30/90-day cards, canvas daily-token chart, top models/users bars
- **Scheduled guest purge** — wipes all guest data daily at an admin-set hour (`guest_purge_hour` , default 00:00) + manual "Purge guest chats now" button
- **Guest Mode** — login-free access via a "Try as guest" button; per-guest message limit (1–200) and session time limit (1–60 min) configurable in Admin Dashboard; live countdown chip; in-app login popup on chats/time exhaustion; server-enforced quota + endpoint lockdown; bilingual (EN/ID); guest data purged on logout
- **Model Connection in Admin Dashboard** — gateway Base URL + API key stored in DB (no more hardcoded), masked key + eye toggle, live Test button (latency + model count), hot-reload, SSRF guard on base URL
- Searchable model combobox, per-user quota edit, realtime quota UI (post-reply + 5 s admin poll), global assistant avatar, custom dialogs (confirm/rename/quota), red model-lock chip, minimized sidebar fixes, versioned asset cache-busting